FHE-STREAM: Characterizing Memory Access Behavior in Fully Homomorphic Encryption Workloads

Ashlesha Malla
Department of Computer Science
Texas Tech University
Lubbock, Texas, USA
asmalla@ttu.edu

Yash Kumar
Department of Computer Science
Texas Tech University
Lubbock, Texas, USA
ykumar@ttu.edu

Michael Beebe
Department of Computer Science
Texas Tech University
Lubbock, Texas, USA
michael.beebe@ttu.edu

Wendy Poole
Los Alamos National Laboratory
Los Alamos, New Mexico, USA
wkpoole@lanl.gov

Yong Chen
Department of Computer Science
Texas Tech University
Lubbock, Texas, USA
yong.chen@ttu.edu

Steve Poole
Los Alamos National Laboratory
Los Alamos, New Mexico, USA
swpoole@lanl.gov

***Abstract*—Fully Homomorphic Encryption (FHE) enables computation on encrypted data, but adoption is limited by high memory overheads. The Residue Number System makes FHE arithmetic practical on CPUs while expanding each polynomial into a fragmented, pointer-based collection of limb arrays. Standard HPC memory benchmarks use flat, contiguous arrays and cannot capture how these structures behave under irregular access. We present FHE-STREAM, a microbenchmark suite that reproduces the memory layout of production RNS-based FHE libraries and measures the gap between the bytes an algorithm intends to move and the bytes the memory system actually transfers. Validation against hardware DRAM counters shows this gap is driven by cache-line utilization rather than access irregularity alone: cache-line-granular or coarser access shows little amplification, while sub-cache-line random access substantially increases traffic. This ordering predicts real cost, with OpenFHE relinearization spending more than half its runtime waiting on memory, providing measured targets for accelerator and library designers.**

***Index Terms*—High-performance computing, Homomorphic encryption, Benchmarking, Memory management, Performance evaluation**

I. Introduction

By enabling complex computations to be performed directly on encrypted data without requiring decryption [1], Fully Homomorphic Encryption (FHE) ensures that sensitive information remains protected even while in active use. This property is directly relevant wherever computation must be outsourced but the data cannot be disclosed, such as encrypted inference over patient records or financial transactions, and multi-tenant cloud and High-Performance Computing (HPC) systems in which the operator is outside the trust boundary. Machine learning over encrypted data is the most developed of these applications [2], [3], and the direction of travel is toward larger models and larger ciphertexts. Instead of relying solely on trusted hardware enclaves, FHE shifts the security paradigm to protect the data itself at every stage of execution.

However, despite its strong theoretical foundations, the commercial and practical adoption of FHE remains bottlenecked by substantial computational and memory overheads [2], [3], [4]. Modern FHE schemes rely on lattice-based cryptography, specifically Ring Learning With Error (RLWE) [5], whose moduli reach thousands of bits. Because fixed-width CPUs cannot operate on such values directly, FHE libraries decompose them using the Residue Number System (RNS) [6], expanding each polynomial into a two-dimensional grid of machine-word limbs. Production libraries such as OpenFHE [7] store that grid not contiguously but as a nested hierarchy of pointers to separately heap-allocated arrays (Section II). Hardware prefetchers mask the resulting indirection during linear sweeps, but real FHE workloads traverse these structures non-contiguously, limiting prefetcher effectiveness [8]. This creates an FHE memory wall, extending the classic architectural bottleneck [9] where memory latency and bandwidth outpace processor speed.

Accurately understanding and quantifying this bottleneck remains a significant challenge. Standard HPC memory benchmarks like STREAM [10] or LINPACK [11] are engineered to stress a system's hardware limits under optimal conditions; however, they do not account for the non-contiguous memory access patterns and software-induced bottlenecks that can be seen in FHE workloads. Conversely, much existing FHE evaluation reports macroscopic metrics such as operations per second, which do not reveal how effectively the memory subsystem is utilized. As specialized accelerators and FHE-focused ASICs such as Intel's Heracles [12] move these operations off general-purpose CPUs, the absence of a common memory-level metric makes cross-platform progress difficult

SC26 Workshops, November 15-20, 2026, Chicago, Illinois, USA
979-8-3195-1221-5/26/$31.00 ©2026 IEEE

to assess. We require precise, scalable metrics to measure progress, expose true hardware bottlenecks, and guide future hardware-software co-design.

To address this critical gap, we introduce FHE-STREAM, a micro-benchmark suite designed to measure memory access behavior under irregular access patterns. We use OpenFHE as our main framework because it is a well-established, widely trusted library in the FHE community, supporting a range of schemes like BFV [13], BGV [14], CKKS [15] and TFHE [16]. Its modular design is especially helpful for our work, since it enables us to cleanly separate the underlying cryptographic components from the higher-level interface, making experimentation and customization much easier. FHE-STREAM adopts the access-pattern taxonomy of RaiderSTREAM [17], but applies it to the pointer-indirected RNS structures that production FHE libraries actually allocate, rather than to flat arrays. A dual-track design separates two costs: one track exercises the underlying limb arrays directly to find the hardware's routing limit, and the second runs on fully instantiated `Ciphertext` objects to measure what the library adds on top of that. Rather than reporting bandwidth alone, we compare the bytes each kernel is required to move against what the memory system actually moves to service it.

This paper makes four contributions. We present FHE-STREAM, a benchmark suite that reproduces the fragmented memory layout of RNS-based FHE rather than the flat arrays assumed by standard HPC memory benchmarks. We validate the suite's reported byte counts against hardware performance counters, showing that the traffic the memory system actually moves is substantially larger than the algorithmic model predicts, and that this gap widens as access becomes less regular. We then show that this ordering tracks the measured cost of real homomorphic operations, establishing that the synthetic kernels are predictive rather than merely illustrative. Finally, we separate the cost of the underlying data movement from the cost added by production object abstractions, by comparing both backends under sequential access with logically-matched payloads.

The remainder of this paper is structured as follows: Section II provides necessary background on lattice-based cryptography, OpenFHE's memory hierarchy, and existing hardware challenges. Section III details the design and methodology behind FHE-STREAM, including our architectural approach and experimental setup. Section IV presents our evaluation and analysis, defining success metrics and analyzing the micro-architectural taxes discovered. Section V reviews related work in FHE acceleration and benchmarking. Finally, Section VI discusses future directions and conclusion.

II. Background

To understand the micro-architectural bottlenecks of FHE, the underlying mathematics must first be understood.

A. Lattice-Based Cryptography and Ring Learning With Error

Modern FHE schemes derive their security from Learning With Errors (LWE), the hardness of recovering a secret vector from linear equations corrupted by small errors. Libraries use its polynomial variant, Ring Learning With Error (RLWE), whose algebraic structure enables the optimizations described below.

In RLWE, data is packed into polynomials in a ring $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$: the ring dimension N fixes the coefficient count per polynomial (chosen as a power of two, up to 65,536 or 131,072 for deep circuits), and Q is the ciphertext modulus. Because FHE operations accumulate noise with every computation, Q must grow with circuit depth, from a few hundred bits for shallow circuits to several thousand for deep inference or bootstrapping [1], the resource-intensive operation used to reset accumulated noise and allow unbounded computation.

B. The Residue Number System and Polynomial Domains

Because Q can span thousands of bits while CPUs operate on 64-bit words, FHE libraries decompose Q using the RNS: a set of L pairwise coprime, machine-word-sized moduli $(q_1, \ldots, q_L)$, each up to 60 bits. Each polynomial is correspondingly split into L independent residue arrays, which we refer to as *limbs* throughout, transforming it into a two-dimensional $N \times L$ grid of 64-bit integers. Because each limb is mathematically independent, RNS also allows homomorphic operations to be vectorized and parallelized across execution pipelines.

Multiplying two polynomials directly in the Coefficient Domain (the array-of-coefficients representation) costs $O(N^2)$, which is impractical at large N. FHE libraries instead use the Number Theoretic Transform (NTT), a finite-field analogue of the Fast Fourier Transform (FFT) that maps operands into an Evaluation Domain, where multiplication is element-wise. Including the forward and inverse transforms, this reduces the total cost to $O(N \log N)$. The system cannot remain in this domain, however: key-switching and noise management require the standard coefficient format, so real FHE workloads repeatedly transform data into the Evaluation Domain via the NTT and back via its inverse (iNTT).

C. The FHE Memory Wall and Hardware Topologies

While RNS and NTT solve the computational problems of FHE, they introduce a memory-layout problem. In production libraries like OpenFHE, the $N \times L$ grid is not stored as a single contiguous block; it is deliberately fragmented, so that individual limbs can be dropped during modulus switching without reallocating the whole polynomial. This flexibility has a cost at the hardware level, described below.

OpenFHE manages these polynomials with the `DCRTPoly` (Double-CRT Polynomial) structure, a wrapper holding scheme metadata and a vector of pointers, each pointing to a separately heap-allocated N-sized limb array. Reading one logical value therefore requires a $1 + 1 + L$ request sequence: wrapper, pointer, payload. This indirection is a one-time lookup that amortizes across the N subsequent reads; what matters is not the indirection itself but its effect on the spatial locality of those reads. Under a linear sweep,

prefetchers lock onto the pattern and stream the limbs at a high fraction of peak bandwidth.

The performance collapse occurs because real-world FHE is dictated by non-linear algorithmic requirements. The most critical operation in FHE is polynomial multiplication, which relies on the NTT. The NTT requires a butterfly memory access pattern, in which each stage pairs coefficients separated by a fixed power-of-two stride and updates them in place. The stride halves at every successive stage, so the access sequence is deterministic and structured rather than random. It is nonetheless hostile to the memory hierarchy: at large ring dimensions the early stages separate paired coefficients by distances far exceeding the cache line and, ultimately, the page. Applied across 20–60 GiB datasets, this long-stride traversal yields poor per-cache-line locality and limits prefetcher effectiveness. Because limbs occupy disjoint heap segments, such traversals may also raise Translation Lookaside Buffer (TLB) pressure and incur page-table walks; we treat this as a contributing rather than a demonstrated factor, as we do not isolate it here.

Software abstraction adds a second layer of cost. Production code operates not on bare `DCRTPoly` elements but on `Ciphertext` objects, each encapsulating at least two `DCRTPoly` structures together with a shared `CryptoContext` holding scheme parameters, evaluation keys, and noise-tracking state.

In practice, then, two effects compound: irregular routing limits prefetcher effectiveness, and the fragmented object layouts used by production libraries add allocation and cache pressure on top of it. Section III describes how FHE-STREAM isolates and measures each of these layers separately.

III. Design & Methodology

This section describes how FHE-STREAM generates workloads, which access patterns it applies, and how its dual-track backend separates hardware routing limits from library-induced overhead.

A. Workload Generation

To guarantee that our benchmarks accurately represent cryptographic stress, FHE-STREAM generates workloads based on three core footprint variables: the Ring Dimension (N), the RNS Depth (L) and the Batch Size. While N and L fix the size of an individual polynomial, the Batch Size represents the number of independent objects processed in a single execution. For the `DCRTPoly` backend, this represents the number of raw polynomial arrays in the test set. In the Ciphertext backend, it represents the number of instantiated ciphertext objects.

For the `DCRTPoly` baseline, we use the `uint64_t` datatype to represent the RNS limbs. We populate these arrays using a Discrete Uniform Generator (DUG) to ensure that every memory location contains random integers distributed uniformly in the range $[0, q_i - 1]$. By using the DUG, we cause the system to perform realistic, full-width modular arithmetic and prevent the compiler from optimizing away trivial zeros. Since the OpenFHE library internally uses `uint64_t` for its RNS limbs, both evaluation tracks maintain a consistent 8-byte width for mathematical data.

The Ciphertext backend is instead populated through OpenFHE's own API. During setup, the system creates a `BFVrns` context and encrypts two packed plaintext vectors: `ct_A` from a vector in which every slot holds the value 1, and `ct_B` from one in which every slot holds 2. We then initialize `ct_C` as clones of `ct_A`. Additionally, we prepare a batch of degree-2 ciphertexts for the relinearization test by pre-multiplying the initial sets. By pre-computing the ciphertexts and isolating the setup phase, the timed execution loop strictly measures the memory bandwidth of the arithmetic operations and data movement. For kernels requiring a scalar, we use a packed plaintext with every slot set to 3.

B. Access Patterns and FHE Translation

To capture the fragmented behavior in Section II, FHE-STREAM ports the four access patterns used in Raider-STREAM to map the physical limits of the system.

- **Sequential:** Represents perfect contiguous streaming, where $A[i] = B[i] + C[i]$. This gives us the best-case baseline for achievable bandwidth.
- **Gather:** Performs random reads from memory, similar to how data is pulled in during complex cryptographic operations.
- **Scatter:** Performs random writes, reflecting how irregular results are written back to memory.
- **Scatter-Gather:** Combines simultaneous random reads and writes. This pattern serves as a worst-case bound on irregular access: because indices are drawn uniformly at random, it admits no prefetch prediction and no reuse of the enclosing cache line.

We include the Scatter-Gather pattern as a bounding case rather than as a model of any specific FHE kernel. It establishes a lower bound on achievable bandwidth under irregular routing, against which real kernels can be positioned. To measure the NTT directly rather than by proxy, FHE-STREAM additionally provides a dedicated NTT Roundtrip kernel, which invokes the library's own forward and inverse transforms. We apply these access patterns at two different levels of granularity for gather, scatter kernels. At the Poly level, we randomize across large, multi-megabyte polynomial blocks. At the Coefficient level, we randomize across individual 64-bit values within the RNS representation, the finer granularity characteristic of core FHE routing.

C. The Dual-Track Benchmarking Strategy

A standard FHE Ciphertext acts as a high-level container, typically encapsulating two or three `DCRTPoly` objects. For instance, at a ring dimension of $N = 131,072$ and depth $L = 40$, a single polynomial requires roughly 40 MiB of memory, bringing a standard degree-1 ciphertext to an 80 MiB mathematical payload. A production Ciphertext operation, however, incurs more than this raw payload.

FHE-STREAM therefore runs every workload through two independent backends: one that operates directly on the underlying limb arrays and one that operates on fully instantiated library objects. Comparing them isolates the cost the library adds over the raw data movement.

1) The DCRTPoly Backend: This track strips away high-level API wrappers to benchmark the underlying 2D grid of RNS limbs directly. It consists of 18 specific kernels designed to measure the raw physical capability of the system under cryptographic routing.

- **Arithmetic Kernels (16):** We execute the standard `COPY`, `SCALE`, `ADD`, and `TRIAD` operations across all four memory access patterns (Sequential, Gather, Scatter, Scatter-Gather).
- **NTT Roundtrip:** Invokes the library's forward and inverse transforms in sequence, measuring the memory behavior of the transform itself rather than of a synthetic proxy.
- **Key-Switch Mock:** Isolates the data-movement penalty associated with evaluation key routing, which dominates homomorphic multiplication.

2) The Ciphertext Backend: This track executes 7 kernels on fully instantiated OpenFHE objects to quantify the cost of object lifecycle management, deep copies, and dynamic allocation. We use the BFV scheme; the object-oriented overheads we measure are common to most schemes.

- **Standard Kernels (4):** We perform the `COPY`, `SCALE`, `ADD`, and `TRIAD` operations. Crucially, these are restricted to Sequential access only. By comparing these sequential results to the sequential `DCRTPoly` baseline, we strictly isolate the software overheads introduced by the object-oriented wrappers.
- **Homomorphic Evaluation (2):** We evaluate `MULT_NO_RELIN` and `RELIN`. Multiplying two degree-1 ciphertexts expands the result to degree-2 (three polynomials rather than two); relinearization reduces it back to degree-1. This expansion-and-reduction cycle is central to the memory behavior of deep workflows such as bootstrapping.
- **In-Place Operations (1):** We evaluate `ADD_INPLACE` to quantify the bandwidth recovered when standard algorithms instead reuse memory for the output objects.

D. Acceleration and Multi-Node Support

FHE-STREAM supports an optional Intel Homomorphic Encryption Acceleration Library (HEXL) [18] backend. This allows the framework to offload performance-critical kernels such as the NTT kernels and modular multiplication to hardware-optimized AVX-512 IFMA (Integer Fused Multiply-Add) instructions, which HEXL leverages. This lets us determine whether the bottleneck is caused by raw arithmetic complexity or by the memory access patterns. We expect that enabling HEXL support will increase performance if FHE operations are not bottlenecked by memory constraints.

For multi-node scalability, FHE-STREAM combines MPI (Message Passing Interface) for process-level distribution across nodes with OpenMP for shared-memory execution within each node, allowing aggregate bandwidth to be measured across multiple memory controllers simultaneously.

TABLE I
CIPHERTEXT BACKEND REPORTED METRICS.

Metric	Definition
PayloadBandwidth	Bandwidth over mathematical payload bytes only
SerializedBandwidth	Bandwidth over live (in-memory) object size
ObjectToPayloadRatio	Serialized size / mathematical payload size
RSS	Resident physical memory, from `/proc/self/statm`

E. Byte Accounting and Bandwidth Definitions

To establish a uniform metric for hardware evaluation, we compute a payload-based bandwidth from the underlying data volume. The size of a single DCRTPoly, denoted as V_{poly}, is calculated as:

$$V_{poly} = N \times L \times 8 \text{ bytes} \tag{1}$$

where N is the ring dimension, L is the RNS depth, and 8 represents the byte-width of a standard 64-bit integer.

For standard two-array operations (e.g., Copy and Scale) that read one array and write to another, the bandwidth (BW) is calculated as:

$$BW = \frac{2 \times B \times V_{poly}}{T_{elapsed}} \tag{2}$$

where B is the batch size. Three-array operations (e.g., Add and Triad) require two reads and one write, scaling the bandwidth calculation to:

$$BW = \frac{3 \times B \times V_{poly}}{T_{elapsed}} \tag{3}$$

Equations (2) and (3) are algorithmic estimates: they count the bytes the kernel is required to move, not the bytes the memory system transfers. Section III-F measures the latter directly and quantifies the gap.

For the Ciphertext backend, we report the metrics summarized in Table I, which separate mathematical throughput from the physical cost of heap fragmentation and object lifecycle management.

As noted in Section III-G, the bandwidth results use the OpenMP `spread` pinning policy so that sequential kernels read from local memory. Under the gather, scatter, and scatter-gather patterns, indices span the full allocation and cross-socket traffic is unavoidable by construction; we report these as measured.

F. Validating the Byte-Accounting Model

The logical byte-accounting model introduced above counts the bytes a kernel is algorithmically required to touch, not the bytes the memory system actually transfers on its behalf. Cache-line granularity, prefetching, and write-allocate policy

all inflate realized DRAM traffic beyond this algorithmic minimum, by an amount that depends on the access pattern. We measure that gap directly with LIKWID [19], reading per-channel DRAM counters (AMD Data Fabric; Intel uncore CAS counts) over the whole process. We define the *amplification factor* as measured bytes divided by algorithmic bytes for the same kernel and direction.

Two limitations apply. Marker-based region instrumentation of this codebase proved unreliable, so we instead measure whole-process DRAM traffic and isolate each kernel by differencing two runs at fixed, widely-separated iteration counts; any cost present in both runs, including fixture setup, cancels out. This yields point estimates rather than distributions, so we report reproducibility directly: each measurement is repeated three times, agreeing within 1.5% in every case, with both sockets populated and balanced. As an independent check, the sequential kernel's measured read amplification ($1.51\times$) matches the $3/2$ ratio predicted by read-for-ownership on a two-read, one-write kernel.

G. Timing Methodology

All timings are wall-clock, taken by Google Benchmark around the kernel loop only; fixture setup (context creation, key generation, encryption) runs once per case, outside the timed region. Each figure is the mean over automatically-repeated iterations, or over independent repetitions with reported coefficient of variation where a single call exceeds the minimum measurement window. Thread affinity is fixed per run using the OpenMP `spread` policy, memory is allocated first-touch, and node allocations are exclusive, with idle DRAM traffic confirmed below 0.03% of theoretical peak.

IV. Evaluation & Analysis

TABLE II
System Information

	AMD Compute Node	Intel Compute Node
CPU	2x AMD EPYC 9754 128 cores/socket 1 thread/core	2x Intel Xeon Gold 6448Y 32 cores/socket 1 thread/core
Main Memory	1.5 TB DDR5, 4800 MHz	512 GB DDR5, 4800 MHz
Peak Bandwidth	921.6 GB/s (858 GiB/s)	614.4 GB/s (572 GiB/s)
Cache Configuration	L1 d/i: 32 KiB, L2: 1 MiB, L3: 512 MiB (total)	L1 d/i: 32 KiB, L2: 2 MiB, L3: 120 MiB (total)
Interconnect	NVIDIA ConnectX-7	
Operating System	Rocky Linux 9.6	
Compiler	GCC 11.5.0	
FHE Library	OpenFHE 1.5.0	
Counter Tool	LIKWID 5.4.1	
FHE Acceleration	None	Intel HEXL 1.2.6

To quantify the micro-architectural bottlenecks of FHE, we conducted benchmarking on the Texas Tech University's REPACSS HPC system. Our evaluation measures memory access behavior during core FHE operations, capturing the impact of both software overhead and irregular data routing. We focused our experiments on two distinct hardware partitions on the cluster due to architectural differences. A detailed breakdown of these systems is provided in Table II.

Our primary testbed is a dual-socket AMD EPYC 9754 node (256 cores, 1.5 TB memory, 512 MiB L3; Table II). Because Intel HEXL is Intel-only, we additionally use a dual-socket Xeon Gold 6448Y node (64 cores, 512 GB memory, 120 MiB L3) with the AVX-512/IFMA support HEXL requires.

Following the cache-busting requirements established in the original RaiderSTREAM methodology, we sized our problem to ensure the CPU could not maintain data residency within the cache. Because the EPYC system has a 512 MiB L3 cache, we utilized a large ring dimension of $N = 131,072$ and an RNS depth of $L = 40$ to represent the large parameter sizes used in practical FHE applications. In FHE, the RNS depth controls the multiplicative noise budget of a ciphertext. A larger depth allows more homomorphic operations to be performed before decryption fails. We chose $L = 40$ because it supports complex workloads such as deep neural network inference and bootstrapping. $N = 131{,}072$ exceeds the minimum for 128-bit security at this modulus size; it is a benchmarking parameter chosen to defeat cache residency. Together these parameters produce a working set an order of magnitude larger than either system's L3 cache while remaining within the depth range used by production inference and privacy-preserving machine learning workloads. For standard three-array operations, a batch size of $B = 512$ generates a working memory footprint calculated as:

$$V_{total} = B \times 3 \times (N \times L \times 8) \approx 60.0 \text{ GiB} \tag{4}$$

This exceeds the AMD L3 by $120\times$ and the Intel L3 by over $500\times$, ensuring cold-RAM rather than cache-resident behavior. While we maintain these N and L values throughout, we adjust the batch size between 512 and 256 depending on the specific backend and experiment to balance hardware saturation with system memory limits.

Since both the `DCRTPoly` and Ciphertext backends use `uint64_t` datatypes, we maintain a standard 8-byte width for all RNS towers throughout the evaluation. Bandwidth results are the mean of ten independent repetitions on an exclusively allocated node; amplification factors are the mean of three, with run-to-run spread below 1.5%. Run-to-run variation is smallest for sequential and regular patterns (CV below 0.4% for Seq Copy and Add) and grows with access irregularity, reaching 5.4% for the irregular Triad kernels and higher for the coefficient-granularity shuffle kernels. This ordering is itself consistent with the mechanism we identify: sub-cache-line access is more sensitive to transient scheduling and allocation state than block-granular access. For our primary evaluations, we utilize the full parallel capacity of each testbed: 256 threads for the AMD nodes and 64 threads for the Intel nodes. We enforced thread affinity via `OMP_PROC_BIND=spread` with

first-touch allocation, so memory pages are resident on the executing core's local RAM.

A. DRAM Traffic Amplification

The gather, scatter, and scatter-gather patterns of Section III-B are designed to stress memory routing; Figure 1 compares the algorithmic byte count of Equations (2)–(3) against the traffic the memory controllers actually move for each, measured per iteration by the differential method of Section III-F.

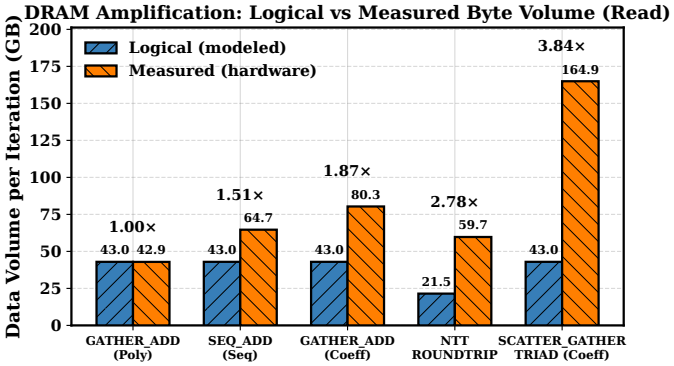


Fig. 1. Logical vs. measured DRAM read volume per iteration, ordered by amplification factor.

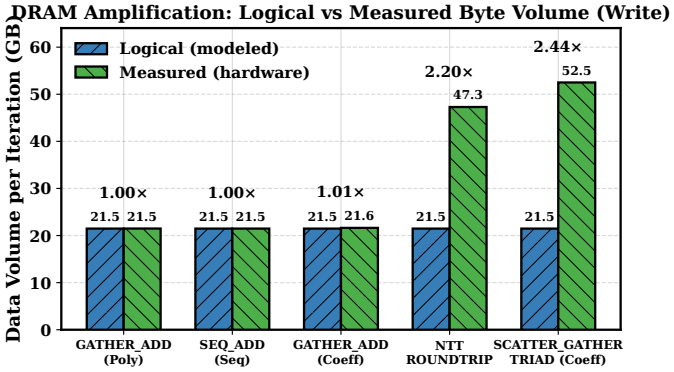


Fig. 2. Logical vs. measured DRAM write volume per iteration.

Observation 1: cache-line utilisation drives amplification, not irregularity as such. Poly-mode gather randomises the order of 1 MiB blocks read sequentially; every fetched line is fully consumed and measured traffic matches the model exactly (1.00×). Randomising the same data at coefficient granularity raises this to 1.87×, as each 8-byte access discards 56 bytes of its line. Granularity alone accounts for the gap.

Observation 2: sequential access is not the floor. `SEQ_ADD` reads 1.51× its requirement while writing exactly one array (1.00×). The excess is read-for-ownership: each output line is fetched before being overwritten, adding one array of reads to the two counted, matching the $3/2$ ratio the measurement predicts. Poly-mode gather is the true floor.

Observation 3: amplification compounds when both directions are sub-cache-line. `SCATTER_GATHER_TRIAD` randomises both ends and is highest in each (3.84× / 2.44×, Figure 2); the NTT round-trip falls between the gather and scatter-gather cases (2.78× / 2.20×), its strided butterfly under-utilising lines without being fully random.

Bandwidth computed from algorithmic byte counts therefore understates memory-system load by up to 3.84×, and most for the patterns FHE depends on. Coarse-grained irregular benchmarks miss this entirely: at the DRAM interface our Poly-mode kernel is indistinguishable from sequential access.

B. Hardware and Routing

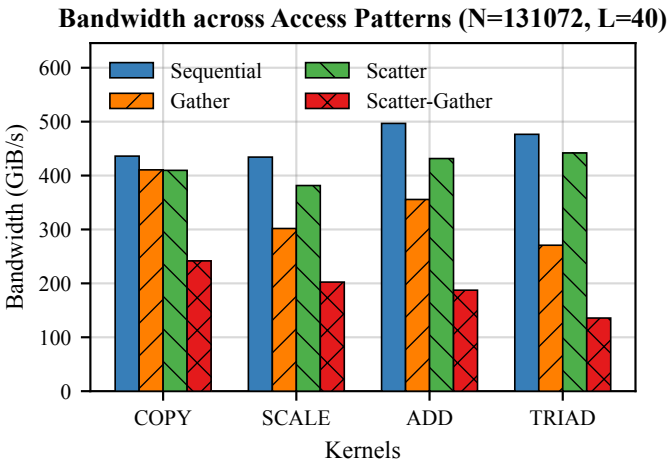


Fig. 3. Measured memory bandwidth across the 4 access patterns on the AMD system.

We used the `DCRTPoly` backend to measure the system's physical memory limits, focusing on its coefficient shuffle mode as the closest representation of production RNS layout; this isolates routing bottlenecks from other memory-management effects. As shown in Figure 3, sequential operations reached 496.7 GiB/s (`ADD`) and 476.5 GiB/s (`TRIAD`) — 57.8% and 55.5% of the 858 GiB/s theoretical peak (Table II) — establishing that FHE is not bottlenecked by raw hardware bandwidth when access is linear. Performance collapses once access becomes irregular: under Scatter-Gather, our worst-case irregular-access bound, `ADD` fell to 187.4 GiB/s (62% loss) and `TRIAD` to 135.7 GiB/s (71% loss); even `COPY` lost 44%. The irregular data movement inherent to lattice-based cryptography degrades performance at the hardware level, independent of software overhead. These figures are algorithmic bandwidth; Section IV-A shows the physical DRAM traffic behind them is 1.00× to 3.84× higher, with the largest gap under exactly the irregular patterns that degrade bandwidth here.

To further understand this routing bottleneck, we compared the performance of Poly and Coefficient shuffle modes. Poly mode randomizes multi-megabyte blocks, while Coefficient mode randomizes individual 8-byte values. As shown in Figure 4, shuffling at the coefficient level results in a much lower performance ceiling. In the Gather Triad kernel, bandwidth drops from 409.7 GiB/s in Poly mode to 278.8 GiB/s in Coefficient mode. The most severe gap appears in the Scatter-Gather

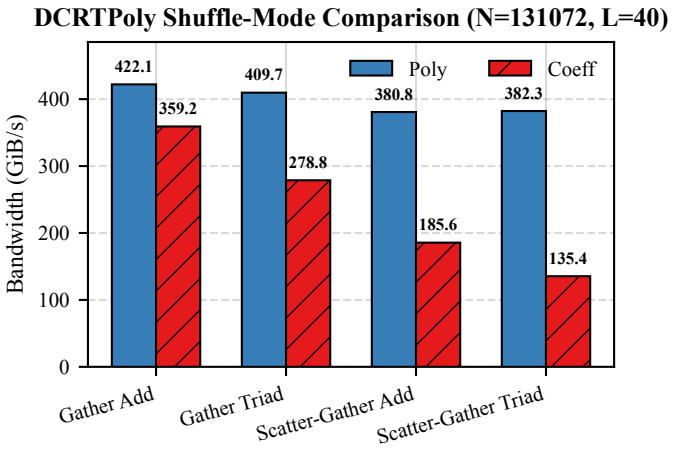


Fig. 4. Measured memory bandwidth comparing the two shuffle modes on the AMD system.

TABLE III
COST OF OPENFHE HOMOMORPHIC OPERATIONS.

Kernel	Operation	Mean (ms)	CV
CT_SEQ_ADD	EvalAdd	176.1	4.6%
CT_SEQ_MULT _NO_RELIN	EvalMultNoRelin	4987.8	2.9%
CT_SEQ_RELIN	Relinearize	32106.0	5.8%

Triad kernel, where performance plummets from 382.3 GiB/s to just 135.4 GiB/s, a 65% reduction. These results indicate that the fine-grained, 8-byte randomness required by FHE operations creates significantly more memory pressure than block-level movement. Section IV-A attributes this gap directly to cache-line utilisation: Poly-mode's multi-megabyte blocks waste no cache-line bandwidth, while Coefficient-mode's 8-byte accesses do, which is why bandwidth and physical DRAM traffic diverge precisely at this granularity.

C. Correlation with Real Homomorphic Operations

To establish that these kernels track production cryptography rather than merely resembling it, we timed three OpenFHE operations at matched parameters ($N = 131{,}072$, $L = 40$, $B = 256$, 256 threads, same node and session, without LIKWID counter instrumentation).

Observation 4: cost follows the amplification ordering. The ratio is $1 : 28 : 182$ (Table III). `EvalAdd` is element-wise over sequentially-traversed limbs, the regime in which we measure the lowest amplification, whereas `EvalMult` and `Relinearize` invoke the NTT and stream evaluation keys, the regime in which we measure the highest amplification ($2.20\times$ to $2.78\times$ for the NTT round-trip).

Observation 5: the costliest operation is stall-bound. `Relinearize` spends 32.1 s wall against 13.9 s CPU ($2.31\times$; 57% stalled), versus $1.18\times$ for `EvalAdd` and $1.42\times$ for `EvalMult`. At $L = 40$ the evaluation-key set exceeds LLC capacity. This is production library code, corroborating the memory-bound result independently of our kernels.

D. Scalability

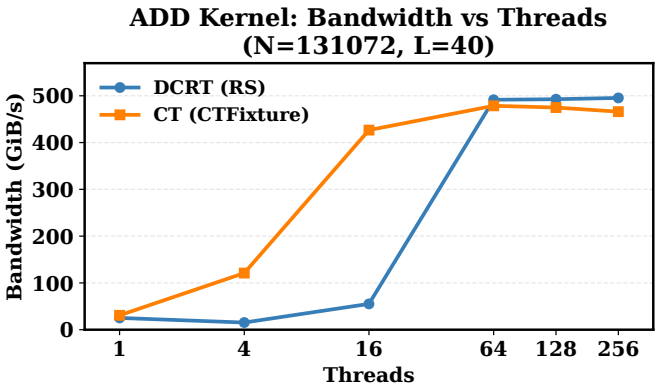


Fig. 5. Measured memory bandwidth with respect to threads used on the AMD system.

We evaluated the parallel efficiency of both backends to identify where software abstractions limit hardware utilization. As shown in Figure 5, `DCRTPoly` ($B = 512$, a 60.0 GiB footprint) requires more threads to saturate bandwidth than the Ciphertext backend ($B = 256$): throughput stays low below 16 threads, insufficient concurrency to keep the memory controllers busy at this footprint, then rises sharply and saturates by 64 threads, whereas Ciphertext rises through 16 threads and plateaus by the same point. The two batch sizes match logical payload (40 MiB per `DCRTPoly`, 80 MiB per degree-1 ciphertext), so the earlier Ciphertext plateau reflects its larger per-thread memory pressure, not an unequal comparison. Because throughput plateaus rather than improving steadily with more threads, both backends are memory-bound: DRAM bandwidth, not CPU core availability, limits the fragmented structures each supplies.

Beyond the single node, we evaluated aggregate bandwidth across an MPI configuration spanning up to 64 nodes. Because each rank operates on an independent batch, the workload distributes across nodes without redistribution of ciphertext data, and aggregate bandwidth grows with the number of memory controllers brought to bear rather than with core count: total throughput reaches 30.9 TiB/s at 16,384 cores. This confirms that the saturation observed above is a per-node property of the memory subsystem rather than a limit of the benchmark itself, and that FHE workloads of this shape can exploit distributed-memory systems provided the working set is partitioned across nodes.

E. Allocation Overhead and In-place Recovery

To isolate the cost of memory management, we compared three versions of the sequential `ADD` kernel at 256 threads. As shown in Figure 6, the standard Ciphertext (`CT`) kernel reached 466.6 GiB/s, a modest drop from the `DCRTPoly` baseline of 493.1 GiB/s, showing that software overhead is manageable under strictly sequential access. The `CT_SEQ_ADD_INPLACE` variant reported 615.9 GiB/s, exceeding the `DCRTPoly` baseline despite performing strictly

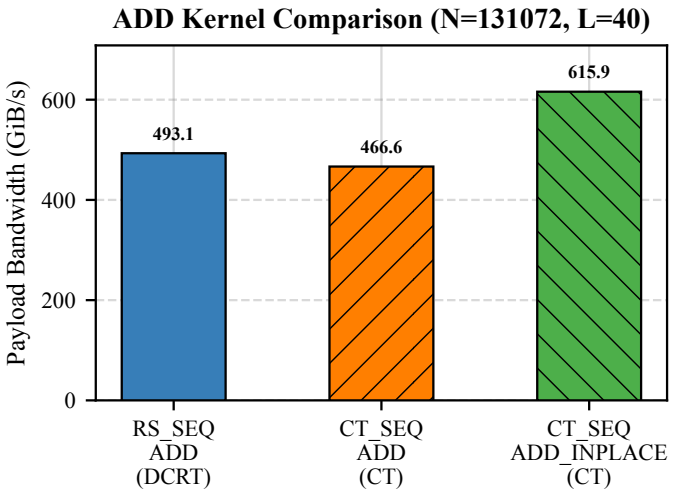


Fig. 6. Measured bandwidth across different Sequential ADD kernels in the AMD system.

more software work. This is a byte-accounting artifact rather than a bandwidth result: the kernel mutates an existing ciphertext instead of constructing a third, so it performs two array traversals while the logical model charges three. The discrepancy is the same one Section IV-A quantifies directly, appearing here as an inflated bandwidth figure rather than an understated one.

F. Memory Expansion and Abstraction Costs

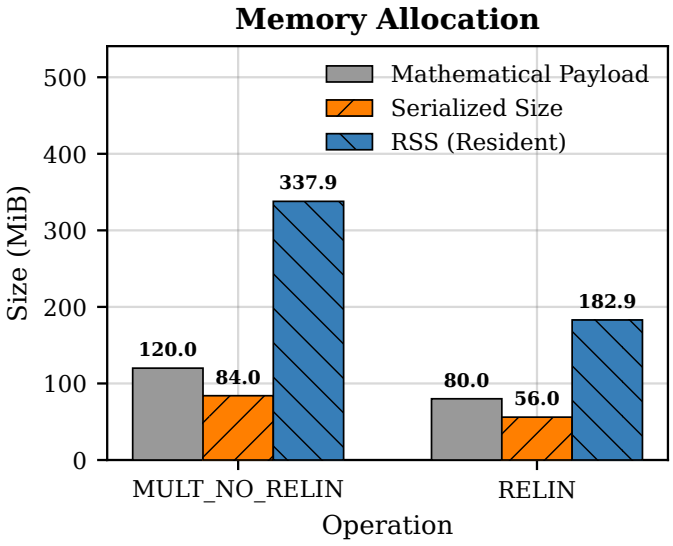


Fig. 7. Memory consumption of Ciphertexts on the AMD system.

Figure 7 compares the mathematical payload of a ciphertext against its resident and serialized sizes, using `RELIN`'s output (a degree-1 ciphertext, 80 MiB) and `MULT_NO_RELIN`'s output (a degree-2 ciphertext, 120 MiB) as reference points. Resident memory exceeds the mathematical payload at both degrees (2.29× and 2.82×), consistent with the allocation and metadata overhead of the object-oriented representation. This overhead grows faster than the underlying mathematics: resident size increases 1.85× from degree-1 to degree-2, against a 1.5× increase in payload, so higher-degree ciphertexts carry proportionally more overhead, not merely more data. Serialized size, by contrast, is a consistent 0.70× the mathematical payload at both degrees, reflecting the compact wire encoding used for storage and transmission rather than the unpacked in-memory representation; the ratio is stable because the same fixed encoding overhead applies regardless of ciphertext degree.

G. CPU-Specific Optimizations

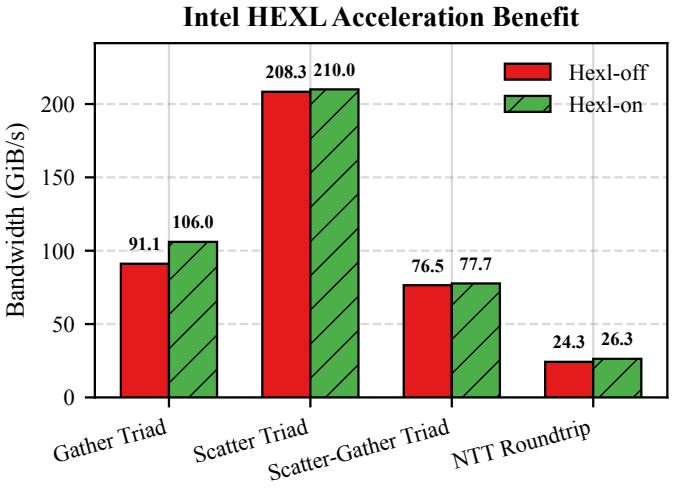


Fig. 8. Measured memory bandwidth with and without Intel HEXL on the Intel system.

To mitigate these computational bottlenecks, vendors have introduced math-centric hardware acceleration, such as the Intel HEXL library, designed to optimize polynomial arithmetic. We compared performance with HEXL-on and HEXL-off on our 64-thread Intel node. As shown in Figure 8, HEXL provides a measurable improvement for two of the four kernels tested: `Gather Triad` rises from 91.1 GiB/s to 106.0 GiB/s, and `NTT Roundtrip` from 24.3 to 26.3 GiB/s. The `Scatter Triad` and `Scatter-Gather Triad` differences are under 2 percent and fall within run-to-run variation, so we do not read them as significant. HEXL therefore accelerates the arithmetic itself, but overall throughput remains constrained by the CPU's ability to fetch data: math-centric libraries are a useful first step, but they do not on their own resolve the irregular data movement inherent to FHE.

V. Related Work

Prior work on the FHE memory bottleneck follows two directions: bypassing the general-purpose CPU, and characterizing it. Accelerator proposals such as F1 [20] and CraterLake [8] report the same qualitative motivation we validate directly in Section IV-A, that irregular RNS access defeats conventional memory hierarchies, and use it to justify large on-chip SRAM and custom routing networks; Processing-in

Memory [21] targets the same bottleneck by moving computation to the memory itself. These works motivate replacing the CPU rather than measuring where, specifically, it falls short. Library-side studies of OpenFHE, SEAL [22], HElib [23], and Lattigo [24] report how memory footprints scale with security parameters and multiplicative depth, but from application-level timing rather than memory-controller measurement, leaving the same gap between algorithmic cost and physical traffic that motivates this work. Standard HPC memory benchmarks such as STREAM [10] and its irregular-access extension RaiderSTREAM [17] characterize hardware limits precisely but on flat, contiguous arrays; neither reports a byte count that can be compared against a cryptographic access pattern's own algorithmic model, which is the comparison our amplification factors provide.

FHE-STREAM occupies the position between these: it retains the general-purpose CPU that accelerator work bypasses, but replaces the application-level timing of library studies with direct memory-controller measurement on the fragmented, pointer-indirected structures those libraries actually allocate. This yields a result neither prior direction reports: a quantified, per-access-pattern gap ranging from no amplification at block granularity to $3.84\times$ at sub-cache-line granularity, and Section IV-C shows this gap predicts the relative cost of real homomorphic operations, giving accelerator designers a CPU-measured target rather than an assumed one.

VI. Conclusion and Future Work

The practical application of FHE is limited by memory bottlenecks, and large, irregularly-accessed polynomial data structures constrain performance before CPU compute limits are reached. FHE-STREAM measures this bottleneck by reproducing the fragmented memory layout of production RNS-based FHE rather than the flat arrays assumed by conventional HPC benchmarks, and by validating its reported byte counts against hardware DRAM counters rather than an algorithmic model alone. That validation shows amplification is governed by cache-line utilization rather than access irregularity in the abstract: access at any granularity coarser than the 64-byte cache line, whether sequential or coarsely shuffled, incurs little amplification, while sub-cache-line random access costs $1.87\times$ to $3.84\times$ the algorithmic byte count. This has a direct implication for accelerator design: on-chip buffering sized to hold whole cache lines, or software restructuring that batches sub-line accesses into line-sized ones, addresses the dominant cost we measure whereas buffering sized only to the logical payload does not. The correlation study further shows this ordering predicts real operation cost. `Relinearize`, the most expensive OpenFHE operation we measured, is also the most stalled ($2.31\times$ real over CPU time), consistent with evaluation-key streaming exceeding last-level cache at high multiplicative depth: a concrete target for key-compression or key-streaming optimizations at deployment-scale depths.

Three directions remain open. All measurements reported here use a fixed parameter point, and the amplification and correlation results specifically are drawn from a single platform, so the magnitudes we report characterize that configuration rather than FHE workloads in general. The underlying mechanism, being a property of cache-line geometry rather than of any particular parameter choice, should generalize more readily than the specific ratios do. First, we plan to expand our kernel families to mimic complete, end-to-end FHE workflows, including profiling full bootstrapping sequences to measure the memory cost of homomorphically resetting accumulated noise. Second, our measurements report aggregate DRAM traffic. Per-cache-level counters would attribute the amplification we observe to a specific level of the memory hierarchy rather than to DRAM alone. Third, we plan to extend the Ciphertext backend to CKKS, BGV, and TFHE, and to repeat these measurements on additional microarchitectures, to test whether the cache-line mechanism identified here generalizes across schemes and platforms.

Acknowledgments

We are thankful to the reviewers for their valuable feedback. We acknowledge that this work is funded in part by LANL/US DoD SOW #19537 and is also supported in part by the National Science Foundation under grant OAC-2404438 and CNS-1939140 (A U.S. National Science Foundation Industry-University Cooperative Research Center on Cloud and Autonomic Computing). The authors used Claude and ChatGPT to assist with proofreading and editing throughout this manuscript and Claude Code was used in some code generation. The ideas and content remain original to the authors.

References

[1] C. Gentry, "A fully homomorphic encryption scheme," Ph.D. dissertation, Stanford University, Stanford, CA, 2009, available online: http://crypto.stanford.edu/craig.

[2] N. Dowlin, R. Gilad-Bachrach, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proceedings of the 33rd International Conference on Machine Learning (ICML-16)*, 2016, pp. 342–351.

[3] E. Hesamifard, H. Takabi, and M. Ghasemi, "Cryptodl: Deep neural networks over encrypted data," 2017. [Online]. Available: https://arxiv.org/abs/1711.05189

[4] A. Sanyal, M. J. Kusner, A. Gascón, and V. Kanade, "Tapas: Tricks to accelerate (encrypted) prediction as a service," 2018. [Online]. Available: https://arxiv.org/abs/1806.03461

[5] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," *Journal of the ACM (JACM)*, vol. 60, no. 6, pp. 1–35, nov 2013.

[6] J.-C. Bajard, J. Eynard, A. Hasan, and V. Zucca, "A full RNS variant of FV like somewhat homomorphic encryption schemes," Cryptology ePrint Archive, Paper 2016/510, 2016. [Online]. Available: https://eprint.iacr.org/2016/510

[7] A. A. Badawi, A. Alexandru, J. Bates, F. Bergamaschi, D. B. Cousins, S. Erabelli, N. Genise, S. Halevi, H. Hunt, A. Kim, Y. Lee, Z. Liu, D. Micciancio, C. Pascoe, Y. Polyakov, I. Quah, S. R.V., K. Rohloff, J. Saylor, D. Suponitsky, M. Triplett, V. Vaikuntanathan, and V. Zucca, "OpenFHE: Open-source fully homomorphic encryption library," Cryptology ePrint Archive, Paper 2022/915, 2022. [Online]. Available: https://eprint.iacr.org/2022/915

[8] N. Samardzic, A. Feldmann, A. Krastev, S. Devadas, K. Eldefrawy, C. Peikert, and D. Sanchez, "Craterlake: A hardware accelerator for efficient unbounded computation on encrypted data," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 173–187.

[9] W. A. Wulf and S. A. McKee, "Hitting the memory wall: implications of the obvious," *SIGARCH Comput. Archit. News*, vol. 23, no. 1, p. 20–24, Mar. 1995. [Online]. Available: https://doi.org/10.1145/216585.216588

[10] J. D. McCalpin, "STREAM: Sustainable memory bandwidth in high performance computers," University of Virginia, Charlottesville, VA, Tech. Rep., 1991–2007, a continually updated technical report. [Online]. Available: https://www.cs.virginia.edu/stream/

[11] J. J. Dongarra, C. B. Moler, J. R. Bunch, and G. W. Stewart, *LINPACK users' guide*. SIAM, 1979.

[12] R. Cammarota, "Intel heracles: Homomorphic encryption revolutionary accelerator with correctness for learning-oriented end-to-end solutions," *Proceedings of the 2022 on Cloud Computing Security Workshop*, 2022. [Online]. Available: https://api.semanticscholar.org/CorpusID:253268334

[13] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," Cryptology ePrint Archive, Paper 2012/144, 2012, https://eprint.iacr.org/2012/144. [Online]. Available: https://eprint.iacr.org/2012/144

[14] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(Leveled) fully homomorphic encryption without bootstrapping," in *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference (ITCS '12)*. New York, NY, USA: Association for Computing Machinery, 2012, pp. 309–325. [Online]. Available: https://doi.org/10.1145/2090236.2090262

[15] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in Cryptology – ASIACRYPT 2017*. Cham, Switzerland: Springer, 2017, pp. 409–437, cryptology ePrint Archive, Report 2017/421.

[16] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "Tfhe: Fast fully homomorphic encryption over the torus," *J. Cryptol.*, vol. 33, no. 1, p. 34–91, Jan. 2020. [Online]. Available: https://doi.org/10.1007/s00145-019-09319-x

[17] M. Beebe, B. Williams, S. Devaney, J. Leidel, Y. Chen, and S. Poole, "Raiderstream: Adapting the stream benchmark to modern hpc systems," in *2022 IEEE High Performance Extreme Computing Conference (HPEC)*, 2022, pp. 1–7.

[18] F. Boemer, S. Kim, G. Seifu, F. D. de Souza, V. Gopal *et al.*, "Intel HEXL (release 1.2)," https://github.com/intel/hexl, 2021.

[19] J. Treibig, G. Hager, and G. Wellein, "Likwid: A lightweight performance-oriented tool suite for x86 multicore environments," in *Proceedings of the 2010 39th International Conference on Parallel Processing Workshops*, ser. ICPPW '10. USA: IEEE Computer Society, 2010, p. 207–216. [Online]. Available: https://doi.org/10.1109/ICPPW.2010.38

[20] A. Feldmann, J. Nikolaidis, J. Karl, and D. Sanchez, "F1: A fast and programmable accelerator for fully homomorphic encryption," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 71–85.

[21] H. Gupta, M. Kabra, J. Gómez-Luna, K. Kanellopoulos, and O. Mutlu, "Evaluating homomorphic operations on a real-world processing-in-memory system," 2023. [Online]. Available: https://arxiv.org/abs/2309.06545

[22] "Microsoft SEAL (release 4.3)," https://github.com/Microsoft/SEAL, Apr. 2026, Microsoft Research, Redmond, WA.

[23] S. Halevi and V. Shoup, "Design and implementation of HElib: a homomorphic encryption library," Cryptology ePrint Archive, Paper 2020/1481, 2020. [Online]. Available: https://eprint.iacr.org/2020/1481

[24] "Lattigo v6," Online: https://github.com/tuneinsight/lattigo, Aug. 2024, ePFL-LDS, Tune Insight SA.

Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

- C_1 FHE-STREAM, a memory microbenchmark suite reproducing the fragmented RNS layouts of production FHE libraries.
- C_2 Validation of algorithmic byte counts against hardware DRAM counters, showing amplification driven by cache-line utilization.
- C_3 Evidence that this ordering tracks the cost of real OpenFHE homomorphic operations.
- C_4 Separation of raw data-movement cost from object abstraction cost.

B. Computational Artifacts

A_1 https://doi.org/10.5281/zenodo.22882262

Artifact	Contributions	Paper Elements
A_1	C_1–C_4	Figs. 1–8, Table III

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

A_1 contains the FHE-STREAM C++ suite, driver scripts for every experiment, the raw data used in the paper, and scripts that regenerate all figures and tables from that data.

Expected Results

On comparable hardware the artifact reproduces the relative trends reported in the paper. Absolute bandwidth and byte counts depend on the memory subsystem.

Expected Reproduction Time (in Minutes)

About 4 hours on an exclusive AMD EPYC 9754 node (Figures 1–7 and Table III), 15 minutes on an exclusive Intel Xeon Gold 6448Y node (Figure 8), and 35 minutes of compute across up to 64 AMD nodes for MPI scaling. Times exclude Slurm queue wait.

Artifact Setup (incl. Inputs)

Hardware: Dual-socket AMD EPYC 9754 (256 cores, DDR5) for all experiments except HEXL, which requires a dual-socket Intel Xeon Gold 6448Y (AVX-512 IFMA). Exclusive node allocation is required. The MPI experiment uses up to 64 AMD nodes.

Software: Rocky Linux 9.6, GCC 11.5.0, OpenFHE 1.5.0, Intel HEXL 1.2.6, Google Benchmark 1.8.5, OpenMPI 4.1.8 (MPI only), CMake $\geq$ 3.5, Python 3 with matplotlib, and LIKWID 5.4.1 built with the access daemon.

Datasets / Inputs: None. Inputs are generated at runtime.

Installation and Deployment: `./setup_env.sh` builds all dependencies. `./reproducibility/build.sh` produces two binaries: a timing binary (all bandwidth and correlation experiments) and a counter binary (DRAM amplification via LIKWID wrapper mode). The HEXL and MPI sbatch scripts build their own binaries on the appropriate nodes.

Artifact Execution

- T_1: DRAM amplification (Figs. 1–2)
- T_2: Bandwidth across access patterns (Fig. 3)
- T_3: Shuffle mode comparison (Fig. 4)
- T_4: Thread scaling (Fig. 5)
- T_5: Sequential ADD comparison and memory allocation (Figs. 6–7)
- T_6: CT correlation (Table III)
- T_7: Intel HEXL comparison (Fig. 8)
- T_8: Multi-node aggregate bandwidth

All tasks use ring dimension $N = 131\,072$ and RNS depth $L = 40$.

Artifact Analysis (incl. Outputs)

Benchmark JSON files, LIKWID counter CSVs, derived amplification factors, and regenerated PDF figures.

Artifact Evaluation (AE)

A. Computational Artifact A_1

Artifact Setup (incl. Inputs)

Three node configurations are required: an exclusive AMD EPYC 9754 node (256 cores) for Figures 1–7 and Table III, an exclusive Intel Xeon Gold 6448Y node (64 cores) for Figure 8, and up to 64 AMD nodes for MPI scaling. Exclusive allocation prevents cross-tenant DRAM traffic from contaminating counter measurements.

Dependencies are OpenFHE 1.5.0, Google Benchmark 1.8.5, Intel HEXL 1.2.6 (Intel node), OpenMPI 4.1.8 (MPI only), CMake $\geq$ 3.5, a C++17 compiler, and LIKWID 5.4.1 with daemon-mode access. The daemon variant is required because the `perf_event` variant cannot program fixed counters on this cluster.

1) Build dependencies: `./setup_env.sh`
2) Activate environment (required before every session): `source env/activate`
3) Load LIKWID daemon module: `module load likwid/5.4.1-daemon`
4) Build binaries: `./reproducibility/build.sh`
5) Verify setup: `./reproducibility/preflight.sh`

Preflight verifies node exclusivity, counter access, environment activation, and binary presence. The HEXL and MPI jobs build their own binaries on the appropriate nodes. No input data is required.

Artifact Execution

All scripts live in `reproducibility/`, set every environment variable explicitly including `RS_BATCH_SIZE` and OpenMP affinity, support a dry-run mode, and write to a timestamped results directory.

```
sbatch reproducibility/run_suite.sbatch
sbatch reproducibility/run_hexl.sbatch
./reproducibility/run_mpi_sweep.sh
```

`run_suite.sbatch` executes all AMD-node experiment groups in sequence. Individual groups can be run selectively with `./reproducibility/run_suite.sh <group>`. Run `./reproducibility/run_suite.sh --help` for available groups and expected runtimes.

Artifact Analysis (incl. Outputs)

Each group writes JSON to its results directory. The plotting scripts in `python_plot/` regenerate each figure from fresh data via `--input`; the HEXL and MPI scripts print the exact command on completion.

Absolute values are platform dependent. On equivalent hardware, amplification factors should agree within 10%, and the CT correlation cost ratio within 5%. MPI scaling should remain near-linear, with the peak set by the interconnect. Thread-scaling curve shape should reproduce even where absolute bandwidth does not. Figure 8 requires an Intel platform with AVX-512 IFMA support.